ComicCraft - AI Comic Story Creator using Gemini Models

Project Description:
ComicCraft is a web-based application that uses AI to generate personalized comic book stories and illustrations based on user-provided prompts. Built with FastAPI and integrated with Google's Gemini AI models along with Stable Diffusion, ComicCraft streamlines the creative process of generating storylines, dialogues, and vivid comic-style imagery automatically.

The application takes in user information—such as story prompt, main character name, setting, tone, and art style—and generates a panel-by-panel storyline and corresponding illustrations. Users can view the comic preview directly on the web interface and also download the full comic in PDF format, structured with both images and narration.

Scenarios
Scenario 1: A user enters a prompt such as "A brave fox exploring an enchanted forest." The system generates a complete comic strip with multiple panels, images and text. The user also specifies the character name, setting, tone and art style.

Scenario 2: A user selects "funny" as the tone and "comic book" as the art style. The system regenerates the entire pipeline—outline, story and images—according to the selected preferences.

Scenario 3: After reviewing the comic preview, the user downloads it. layout_builder.py assembles the panels and exporters.py compiles the comic into a PDF using FPDF.

Technical Architecture

Description: ComicCraft is structured into three primary components: Frontend (HTML, CSS, Jinja2), Backend (FastAPI application), and AI Integration (Google Gemini APIs and Hugging Face Diffusers).

The frontend captures user inputs and displays generated comics. The FastAPI backend manages routing, data processing and template rendering. Gemini models generate story content and Stable Diffusion generates comic-style illustrations.

Frontend: index.html collects story prompt, character name, setting, tone and art style. comic_preview.html displays the generated comic and export_success.html shows export confirmation.

Backend: FastAPI receives form or JSON data, calls AI generation functions, builds the comic layout and exports the PDF.

AI Integration: Gemini Flash creates the structured five-panel outline, Gemini Pro creates narration and dialogue, and Stable Diffusion generates illustrations.

Pre-requisites & Project Workflow

Pre-requisites:

1. FastAPI Framework Knowledge
2. Uvicorn ASGI Server
3. Python Programming Proficiency
4. HTML, CSS, and Jinja2 Template Skills
5. Google Gemini API Familiarity
6. Hugging Face Diffusers & Stable Diffusion
7. PyTorch Basics
8. PDF Generation with FPDF
9. Version Control with Git
10. Python Package & Environment Setup (pip, virtualenv, or conda)

Project Workflow:

Milestone 1: Model Selection and Architecture
Activity 1.1: Research and Select the Appropriate Generative AI Model
Activity 1.2: Define the architecture of the application
Activity 1.3: Set up the development Environment

Milestone 2: Core Functionalities Development
Activity 2.1: Develop the core functionalities
Activity 2.2: Implement the FastAPI Backend to Manage Routing and User Input Processing

Milestone 3: routes.py Development
Activity 3.1: Writing the Main Application Logic in routes.py

Milestone 4: Frontend Development
Activity 4.1: Designing and Developing User Interface
Activity 4.2: Creating Dynamic Templates with FastAPI’s Jinja2

Milestone 5: Deployment
Activity 5.1: Preparing the Application for Local Deployment
Activity 5.2: Testing and Verifying Local Deployment

Milestone 1: Model Selection and Architecture

In this milestone, appropriate generative AI models are selected for ComicCraft’s requirements of generating comic storylines, character dialogues, scene descriptions and comic-style illustrations.

Activity 1.1: Research and Select the Appropriate Generative AI Model

Understand the Project Requirements: The application must generate structured comic outlines, full comic narration and character dialogues, and visual comic-style illustrations for each panel.

Model Evaluation: Google Gemini Flash is optimized for fast structured outputs. Gemini Pro is used for detailed creative text generation. Hugging Face Diffusers with Stable Diffusion is used for image generation.

Models are considered based on response quality for creative writing, prompt adherence, scene-detail richness, image generation speed and resolution, stability, documentation and community support.

Activity 1.1: Final Model Selection

Gemini Flash (models/gemini-1.5-flash): Selected for fast and structured generation of comic panel outlines based on story prompts.

Gemini Pro (models/gemini-1.5-pro): Selected for detailed, creative comic narration and engaging character dialogues.

Stable Diffusion (runwayml/stable-diffusion-v1-5): Selected for generating comic-style illustrations from image prompts.

This combination allows ComicCraft to deliver creative storytelling alongside visually appealing comic panels in an automated workflow.

Activity 1.2: Define the architecture of the application

Architecture Overview: The architecture consists of Frontend (HTML, CSS, Jinja2), Backend (FastAPI application), and AI Integration (Google Gemini APIs and Hugging Face Diffusers).

Frontend Responsibilities: Users interact with a simple web interface. Inputs include Story Prompt, Character Name, Setting, Story Tone and Art Style. The form sends data to FastAPI using POST.

Templates Used: index.html for input collection, comic_preview.html for generated comic display, and export_success.html for successful export confirmation.

Backend Responsibilities: FastAPI receives form/JSON input, calls AI models, organizes comic layout and exports the comic as PDF.

Activity 1.2: AI Integration Points

Gemini Flash: Generates the structured 5-panel comic outline based on the user’s story prompt.

Gemini Pro: Creates detailed narration and character dialogue from the outline.

Stable Diffusion: Generates comic-style illustrations from image prompts for each panel.

The backend formats the AI responses and passes the generated text and images to the frontend templates for preview and export. Each comic is generated dynamically from the user’s inputs.

Activity 1.3: Set up the development Environment

Install Python and Pip and create a project-specific virtual environment.

Windows:
python -m venv comiccraft-env
comiccraft-env\Scripts\activate

Install Required Libraries:
pip install fastapi uvicorn jinja2 python-multipart google-generativeai diffusers transformers fpdf Pillow accelerate

Run Server:
uvicorn app.main:app --reload

Visit http://127.0.0.1:8000 for the application and /docs for API testing.

Milestone 2: Core Functionalities Development

Activity 2.1: Develop the core functionalities

Generate Structured Comic Panel Outline
Function: generate_outline()
File: gemini_flash.py

This function uses Gemini Flash to generate a structured 5-panel comic outline. Each panel contains a panel number, title, scene description and image generation prompt. The output is a list of dictionaries, one for each panel.

Generate Comic Story Narration and Dialogue

Function: generate_story()
File: gemini_pro.py

This function uses Gemini Pro to expand the panel outlines into a full comic-style story. It creates engaging narration and character dialogues for each panel and returns formatted story content.

Generate Comic Illustrations from Prompts & Organize Comic Panels and Export PDF

Function: generate_image()
File: image_generator.py

This function uses Stable Diffusion to create a comic-style image based on the provided image prompt. It sanitizes the prompt for safe filenames and saves the generated image to the static/panels directory. It returns the saved image path.

Function: build_comic_layout()
File: layout_builder.py

This function organizes generated images and the full comic story into a structured layout and matches each image with its corresponding panel story.

Function: save_pdf()
File: exporters.py

This function compiles the full comic into a multi-page PDF using FPDF. Each panel’s image and narration are placed neatly on separate pages. The final PDF is saved into the static/exports folder.

Activity 2.2: Implement the FastAPI Backend

All routing logic is handled in routes.py. Each route is linked to the corresponding backend function that processes the comic-generation workflow.

Process User Input: HTML form in index.html collects story prompt, main character name, setting, tone and art style. FastAPI captures these using Form(...) parameters. PromptRequest validates JSON input for API-based generation.

/generate: Calls generate_outline(), generate_story(), generate_image() for each panel, build_comic_layout(), and save_pdf().

/generate-comic/json: Uses a similar workflow for raw JSON requests and returns layout data and PDF path.

/test-image: Calls generate_image() directly to test image generation.

Milestone 3: routes.py Development

Milestone 3 focuses on setting up the major FastAPI routes, handling user inputs, integrating AI generation workflows and ensuring interaction between backend processing and frontend templates.

Activity 3.1: Writing the Main Application Logic in routes.py

/ Loads the homepage index.html where users submit story details.

/generate Handles form submission, processes input using AI, generates comic panels and returns the comic preview.

/generate-comic/json Accepts JSON payloads, triggers comic generation and returns layout data and PDF path.

/export-success Displays success confirmation after comic export.

/test-image Tests image generation from a direct prompt.

Route Handlers and AI Workflows

Set up Route Handlers: /generate captures Story prompt, Character name, Setting, Story tone and Art style. /generate-comic/json captures a PromptRequest JSON body.

Route handlers process the data through generate_outline, generate_story, generate_image, build_comic_layout and save_pdf.

Error Handling: FastAPI HTTPException and try-except blocks are used to handle unexpected API errors or JSON decode errors.

Integrate AI workflows: The generated outputs are passed to comic_preview.html or returned as JSON for API clients.

Milestone 4: Frontend Development

Milestone 4 focuses on building responsive and clean web pages using HTML, CSS and Jinja2. Users can submit inputs, view generated comics panel-by-panel and confirm export actions.

Activity 4.1: Designing and Developing the User Interface

Set Up the Base HTML Structure: index.html is the main entry point for user input. The form captures Story Prompt, Main Character Name, Setting, Story Tone and Art Style.

Design a Responsive Layout Using CSS: CSS styles provide centered layouts, consistent buttons, hover effects, padding, box shadows and rounded corners.

Create Separate Pages for Each Core Functionality

index.html: A user-friendly form where users input the story prompt, character name, setting, tone and preferred art style. It submits data to the /generate FastAPI route via POST.

comic_preview.html: Displays the AI-generated comic panel-by-panel. Each panel includes title, scene illustration, scene description, captions and narration.

export_success.html: Displays a success message after the user downloads the comic as a PDF and encourages the user to create another comic.

Templates are modular and separated by functionality, providing a smooth flow from comic creation to review and export.

Activity 4.2: Creating Dynamic Templates with FastAPI’s Jinja2

Integrate Jinja2 Templating: FastAPI’s Jinja2Templates is used for dynamic content rendering.

Each FastAPI route uses TemplateResponse: / renders index.html, /generate renders comic_preview.html, and /export-success renders export_success.html.

Bind Backend Data to HTML Templates: index.html form elements map to backend Form(...) parameters. comic_preview.html displays panel number, title, generated image, scene description, captions and narration. The layout is rendered using Jinja2 loops.

The frontend is fully integrated with FastAPI and dynamically renders AI-generated comic content.

Milestone 5: Deployment

Milestone 5 focuses on deploying the ComicCraft application locally using FastAPI. This involves configuring the server environment, installing dependencies, setting up static files and templates, and launching the application locally.

Activity 5.1: Preparing the Application for Local Deployment

Set Up a Virtual Environment:
python -m venv env
env\Scripts\activate (Windows)
source env/bin/activate (macOS/Linux)
pip install -r requirements.txt

Configure Environment Variables: Use environment variables for the Gemini API key and Hugging Face API token.

GEMINI_API_KEY=your_gemini_api_key_here
HF_API_KEY=your-huggingface-api-key-here

Activity 5.2: Testing and Verifying Local Deployment

Start the FastAPI Server: Navigate to the project root and run:
uvicorn app.main:app --reload

Access the Application Locally: http://127.0.0.1:8000

The homepage allows users to input story prompt, character name, setting, tone and art style to generate a personalized AI-powered comic.

FastAPI Documentation: http://127.0.0.1:8000/docs

Available endpoints include /generate-comic/json, /generate, /export-success and /test-image.

Exploring the Website’s Web Pages

Home Page: The homepage is an intuitive starting point for creating AI-generated comic stories. It provides a structured form for Story Prompt, Main Character Name, Setting, Story Tone and Art Style.

Once submitted, the system generates storyline outlines, detailed dialogues, illustrations and an exportable PDF.

Comic Preview – Result Page: The Comic Preview Page presents generated comic panels sequentially after the user submits the form.

Comic Preview, Download Button and Conclusion

Comic Preview Page: Each panel includes Panel Title, Comic Image, Scene Description, Caption and Narration, and Image Prompt Reference.

Download Button: “Download Your Comic as PDF” compiles the complete comic including images, panel titles, scene descriptions, captions and narration into a PDF file.

Comic Export Success Page: After download, the user is redirected to a confirmation message and can choose “Go Create Another Comic”.

Conclusion: ComicCraft combines Gemini Flash and Gemini Pro for storytelling and Stable Diffusion for comic-style illustration. It provides an end-to-end experience from personalized story outline generation, narration and image creation to downloadable PDF export.